

Cryptographic Migration Engineering

Ein simulationsbasiertes Oracle zur sicheren Orchestrierung
der Post-Quantum-Kryptografie-Migration

YAML Twin → Oracle → Deploy → Verify

Erick Zeiler

Masterprojekt Informatik · Hochschule RheinMain

Prof. Dr. Marc Stöttinger · Prof. Dr. Bodo Iglar

Motivation und Forschungsfrage

Warum PQC-Migration?

- Quantencomputer bedrohen RSA und ECC (Shor-Algorithmus).
- NIST hat Post-Quanten-Kryptografie standardisiert (ML-KEM, ML-DSA).
- Migration ist kein Algorithmus-Tausch, sondern ein Systemproblem.

Zentrale Forschungsfrage

„Können versteckte kryptografische Abhängigkeiten automatisch erkannt und in eine sichere Migrationsreihenfolge überführt werden?“

Das Problem

- Server ↔ Client (Key Exchange)
- Zertifikat ↔ Schlüssel (Trust Chain)
- TLS-Version ↔ KEX-Algorithmus

Naive Einzel-Migration eines Assets kann das gesamte System ausfallen lassen.

Klassisch (RSA, ECDHE) → Post-Quanten (ML-KEM, ML-DSA)

Post-Quanten-Standards (NIST)

Standardisierte Algorithmen

- **ML-KEM** (FIPS 203) — Key Encapsulation, ersetzt ECDHE/DH
- **ML-DSA** (FIPS 204) — digitale Signaturen, ersetzt RSA/ECDSA
- NIST-Standardisierung 2024 — Migration wird verpflichtend

Hybrid-Übergang

- Nicht Big-Bang, sondern schrittweise Umstellung
- **X25519_MLKEM768** — klassischer + PQC-KEX parallel
- Server und Client müssen denselben Modus sprechen

CRYME-Migrationspfad (PoC)

Phase	Asset	Ziel
1	Key Exchange	X25519_MLKEM768
2	Zertifikat	ML-DSA-44
3	TLS-Policy	1.3 only

Warum relevant?

Jeder Schritt ändert das Verhalten des Live-Systems — falsche Reihenfolge oder isolierte Migration → Ausfall

Was ist CRYME?

CRYME als Orchestrator

CRYME (Cryptographic Migration Engineering) plant, validiert und deployt PQC-Migrationen. Es ersetzt keinen TLS-Stack.

YAML Twin → Oracle → Deploy → Verify

Kernkommandos

```
$ cryme init
$ cryme migrate ...
$ cryme deploy step=N
$ cryme show tree
```

Drei Mechanismen

- **Dependency Discovery:** implizite Kanten → Co-Migrations-Cluster (SCC)
- **Temporal Barriers:** keine Out-of-Order-Zustände, deny-by-default
- **Event Sourcing:** Migrationsbaum mit Replay zu jedem Schritt N

Prinzip

„**Prüfen vor Ausführen** — jeder Migrationsschritt wird simuliert und validiert, bevor er auf dem realen System ausgeführt wird.“

Architektur und Domänenmodell

Komponenten

- **cryme CLI** — Orchestrator (planen, validieren, deployen)
- **Memgraph** — Digital Twin + Migrationshistorie
- **oracle.js** — SCC, implizite Kanten, Temporal Rules
- **nginx :8443** — live migrierbarer HTTPS-Dienst

Digital Twins (YAML)

- Components: Webserver, Browser
- CryptoAssets: Zertifikate, Key Exchange
- SecurityControls: TLS-Version
- PQC Variants: ML-KEM, ML-DSA

PoC-Szenario: `webserver_pqc_twin.yaml`

Pipeline

YAML → cryme migrate → Oracle → cryme deploy → Ansible
→ nginx :8443

Oracle-Regeln

1. SCC-Expansion — Cluster gemeinsam migrieren
2. Temporal Barriers — Vorbedingungen prüfen
3. Deny-by-default — unsichere Schritte → FAILED

Tech-Stack & Implementierung

Anwendungsschicht

- **Node.js** — cryme CLI + Oracle-Engine
- **oracle.js** — SCC (Tarjan), Migration, Event Replay
- **twin_loader.js** — YAML → Memgraph
- **neo4j-driver** — Bolt-Protokoll (:7687)
- **js-yaml** — Digital-Twin-Parsing

Infrastruktur (Docker)

- **Memgraph** — Graph-DB + Migrationshistorie
- **Memgraph Lab** — Graph-Visualisierung (:3000)
- **nginx 1.27** — TLS-Termination (:8443)
- **curl-client** — Browser-Simulation
- **docker-compose** — gesamter Live-Stack

Konfiguration & IaC

- **YAML** — Digital Twins (webserver_pqc_twin.yaml)
- **Ansible** + Jinja2 — Rolle cryme_tls
- Generierte Playbooks in playbooks/

Verifikation

- **OpenSSL** — TLS-Handshake-Analyse
- **curl** — /api/status + Cipher-Suite-Check
- **verify_tls.sh** — unabhängig von cryme API
- Zwei Sichten: Graph-Zustand ↔ Wire-Verhalten

Oracle-Logik im Detail

Drei Kantentypen

- **Explizit** — modelliert in YAML (depends_on)
- **Implizit** — z. B. SecurityControl → CryptoAsset
- **Discovered** — beim FAIL neu gelernt (Server ↔ Client)

Temporal Barriers

- not_before-Constraints zwischen Komponenten
- Beispiel: TLS 1.3 only erst nach abgeschlossenem KEX
- Verhindert unsichere Zwischenzustände → ABORTED

SCC-Clustering (Tarjan)

- Starke Zusammenhangskomponenten im Graphen
- Knoten in einer SCC → **Co-Migration** Pflicht
- Ein Befehl, ein Cluster — Oracle erweitert automatisch

Policy-Modell

- **Deny-by-default** — unsicherer Schritt → FAILED
- Simulation **vor** Deploy — kein Schaden am Live-System
- Event Sourcing — jeder Versuch im Migrationsbaum

Demo-Setup

Live-System (ilmare)

- cryme-memgraph — Graph-DB
- cryme CLI — Oracle
- cryme-nginx-classic — HTTPS :8443
- cryme-curl-client — Browser-Simulation

Migrationsziel

Schrittweise Umstellung: Hybrid-KEX → ML-DSA → TLS

1.3 only

Verifikation unabhängig via curl und openssl

Baseline · Schritt 0

Zertifikat	RSA-2048
KEX	ECDHE (Server + Client)
TLS	1.2 / 1.3
Profil	classic-rsa-ecdh

Zwei Sichten — eine Wahrheit

- **Planung:** cryme show state step=N
- **Laufzeit:** API + TLS-Handshake auf Port 8443
- Nach Deploy stimmen Graph-Zustand und nginx überein.

Demo: Schritt 1 — Versteckte Abhängigkeit (FAIL)

```
$ cryme migrate id=Webserver__Classic.KeyExchange__ECDHE  
X25519__MLKEM768  
FAILED — Policy Denied  
Server KEX migrated to PQC, but  
Client__Browser.KeyExchange__ECDHE is still Classic.
```

Das Client Problem

- Oracle erkennt implizite Kommunikationskante Server ↔ Browser.
- Server-KEX allein migrieren → Client kann nicht mehr kommunizieren.
- Schritt wird präventiv blockiert (deny-by-default).

Kernaussage

CRYME erkennt versteckte Abhängigkeiten, bevor sie auf dem Live-System Schaden anrichten.

Demo: Schritt 2 — Co-Migration (SUCCESS)

SUCCESS — Co-Migrated Cluster

Webserver.KeyExchange_ECDHE → X25519_MLKEM768

Client_Browser.KeyExchange_ECDHE → X25519_MLKEM768

\$ cryme deploy step=2

Gleicher Befehl — anderes Ergebnis

- Oracle erkennt SCC-Cluster und migriert beide Endpunkte gemeinsam.
- Profil wechselt zu `hybrid-kex-classic-cert`.
- API und TLS-Handshake bestätigen Hybrid-KEX unabhängig.

Thesis-Nachweis

Versteckte Abhängigkeit erkannt (FAIL) → Co-Migration erlaubt (SUCCESS) → Live-Deploy verifiziert.

Migrationsbaum und weitere Schritte

```
$ cryme show tree
[Step 0] INIT
├── [Step 1] FAILED
├── [Step 2] SUCCESS (Co-Migrated)
│   ├── [Step 3] ABORTED
│   └── [Step 4] SUCCESS (Cert)
│       └── [Step 5] SUCCESS (TLS 1.3)
```

Schritte 3–4

- **Schritt 3:** Zertifikat RSA → ML-DSA-44
- **Schritt 4:** TLS 1.2/1.3 → TLS 1.3 only
- Jeweils: migrate → deploy → verify

Event Sourcing

- Jeder Versuch wird dokumentiert — auch Fehlschläge.
- Zustand bei Schritt N durch Replay rekonstruierbar.
- Git-Analogie: Commit = MigrationStep · log = tree · HEAD = Zustand

Ergebnisse

Thesis-Claims nachgewiesen

- ✓ Versteckte Abhängigkeiten erkannt (Schritt 1 FAILED)
- ✓ SCC-Co-Migration funktioniert (Schritt 2 SUCCESS)
- ✓ Live-Deploy auf HTTPS (API + TLS ändern sich)
- ✓ Unabhängige Verifikation (curl / openssl)

TLS-Profile (cryme_tls)

Profil	KEX	TLS
classic-rsa-ecdhe	ECDHE	1.2+1.3
hybrid-kex-classic-cert	X25519_MLKEM768	1.2+1.3
hybrid-kex-mldsa-cert	X25519_MLKEM768	1.2+1.3
tls13-only	beliebig	1.3 only

Neue Algorithmen über `migration_variants` in YAML — ohne Code-Änderung

Grenzen, Fazit und Ausblick

PoC-Grenzen

- Ein Szenario (Webserver + Browser)
- ML-DSA-Zertifikate experimentell
- Keine vollautomatische Migrationsplanung
- Operator gibt Migrationsschritte vor

Vision

- Beliebige kryptografische Landschaften
- Automatisierte PQC-Iterationsplanung
- CI/CD-Integration: Crypto-as-Code

Fazit

CRYME demonstriert: Riskante PQC-Migrationsschritte lassen sich **vor** der Ausführung simulativ erkennen und abfangen.

„Versteckte Abhängigkeiten → Clusters → sichere Reihenfolge“

Beitrag

1. Oracle-Framework für kryptografische Migration
2. Graphbasiertes SCC-Clustering
3. End-to-End: YAML → Deploy → Verify
4. Event Sourcing & Audit-Trail

Vielen Dank für die Aufmerksamkeit

Fragen?

Live-Demo: cryme init → cryme migrate → cryme deploy
/cryme · <https://127.0.0.1:8443>